

CHALLENGE #15: CaaS Write-up.

caas

Medium

Web Exploitation

picoMini by redpwn

AUTHOR: BROWNIEINMOTION

Hints ?

Description

(None)

Now presenting [cowsay as a service](#)

CHALLENGE ENDPOINTS

Download index.js

[index.js](#)

22,045 users solved

97%

Liked

picoCTF{FLAG}

Submit Flag

Solving Process:

The website appeared to be a “Cows as a Service” application, which generates a message spoken by a cow. The page allowed users to enter a message through the URL, and the cow would display the text as part of its output. This behavior resembled the well-known cowsay command-line tool, where a cow prints a user-provided message.

Kali Linux [Running] - Oracle VirtualBox

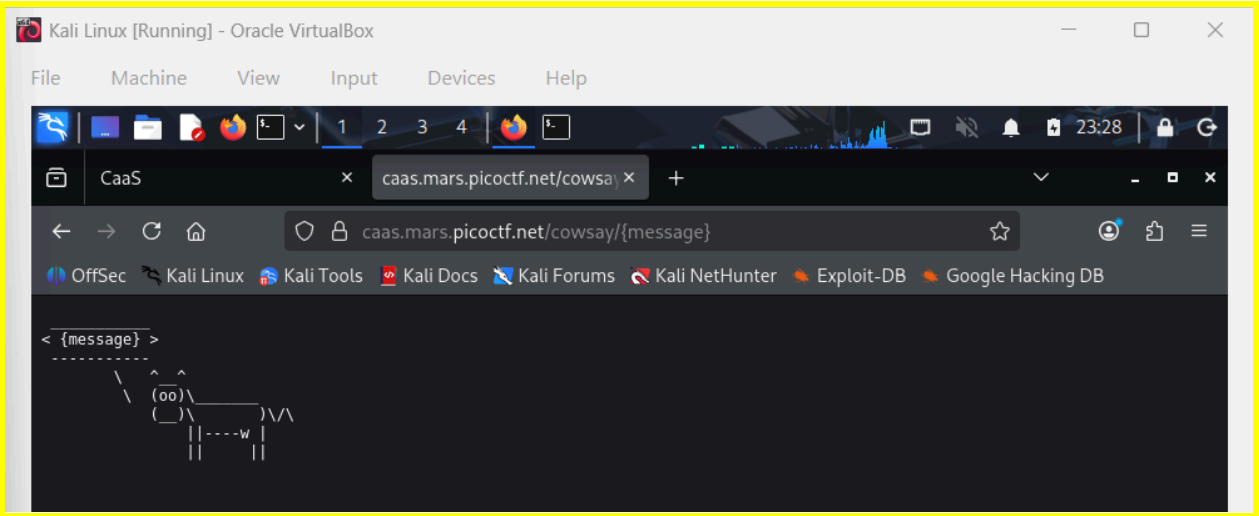
FileMachineViewInputDevicesHelp

CaaS

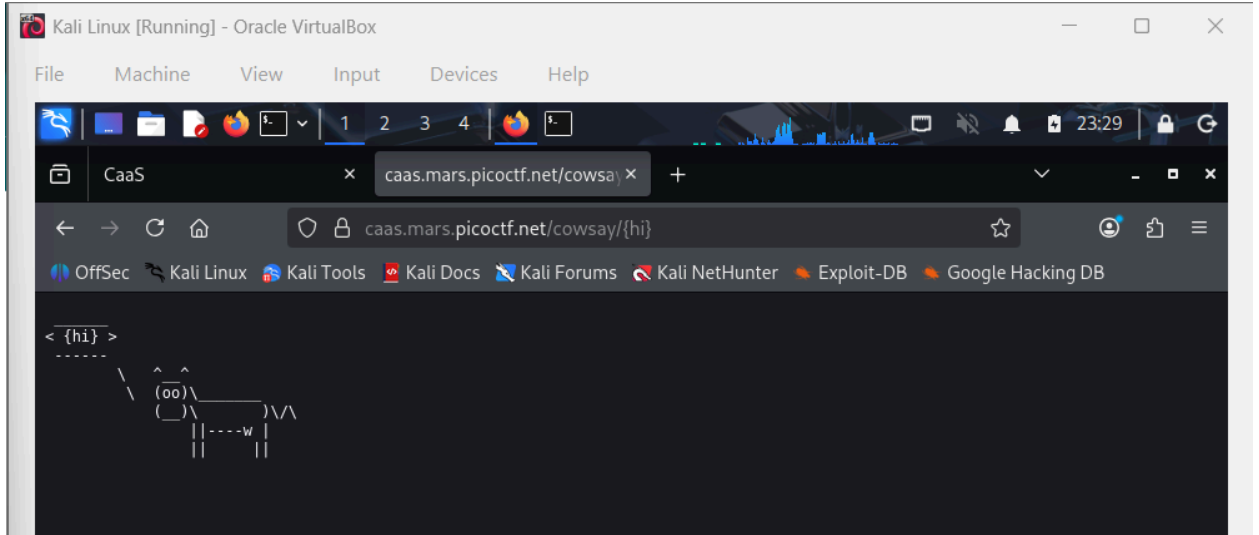
caas.mars.picoctf.net

Cowsay as a Service

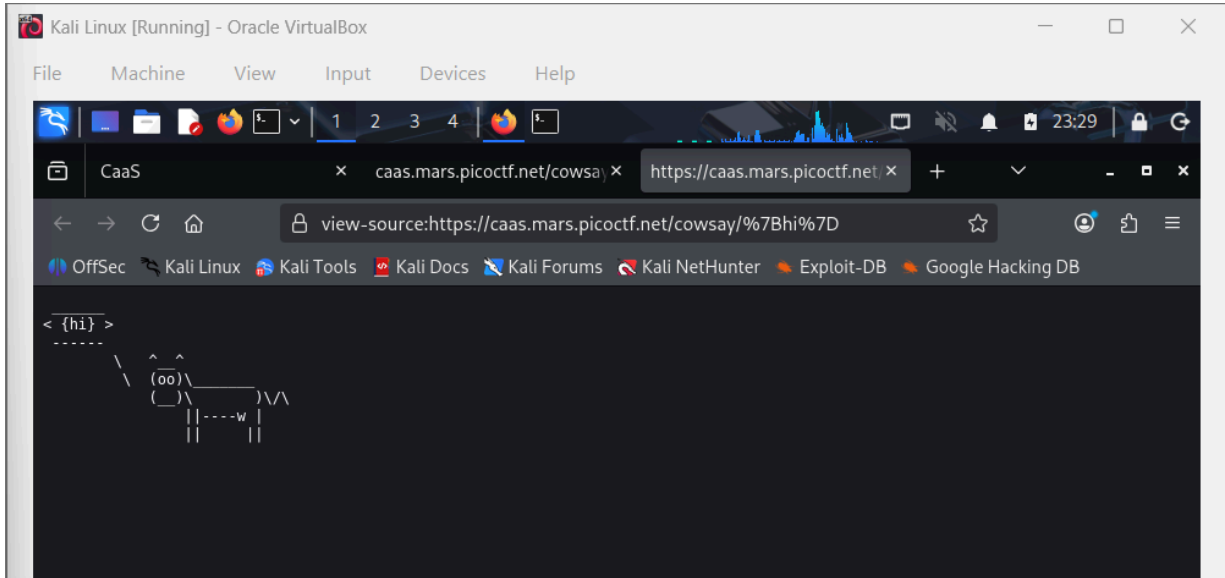
Make a request to the following URL to cowsay your message:
https://caas.mars.picoctf.net/cowsay/{message}



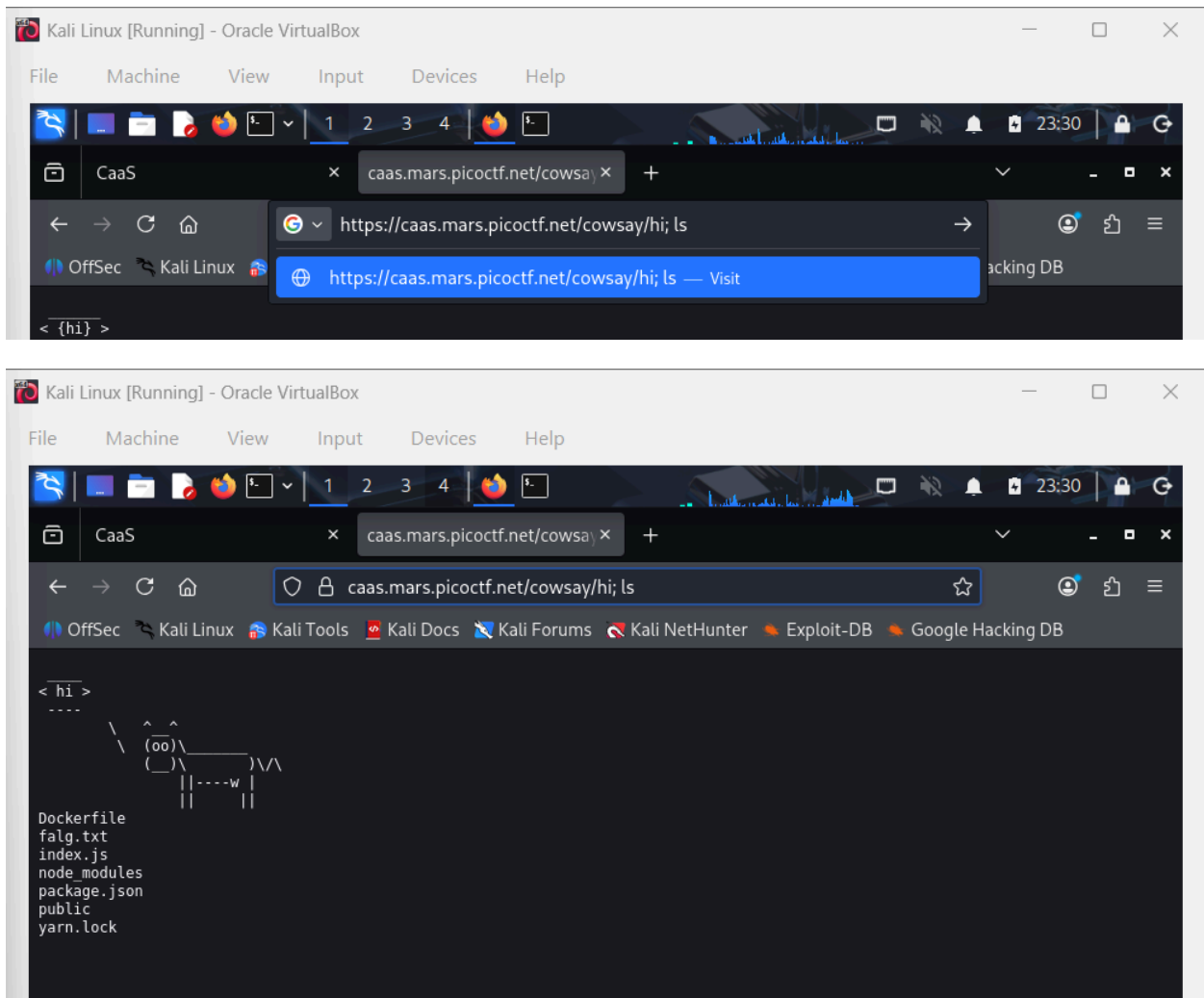
To understand how the application worked, the provided URL was opened in another tab and a simple message was tested. By modifying the message parameter in the URL and entering a word such as **“hi”**, the page successfully displayed the cow saying the provided message. This confirmed that the website was dynamically processing the input from the URL.



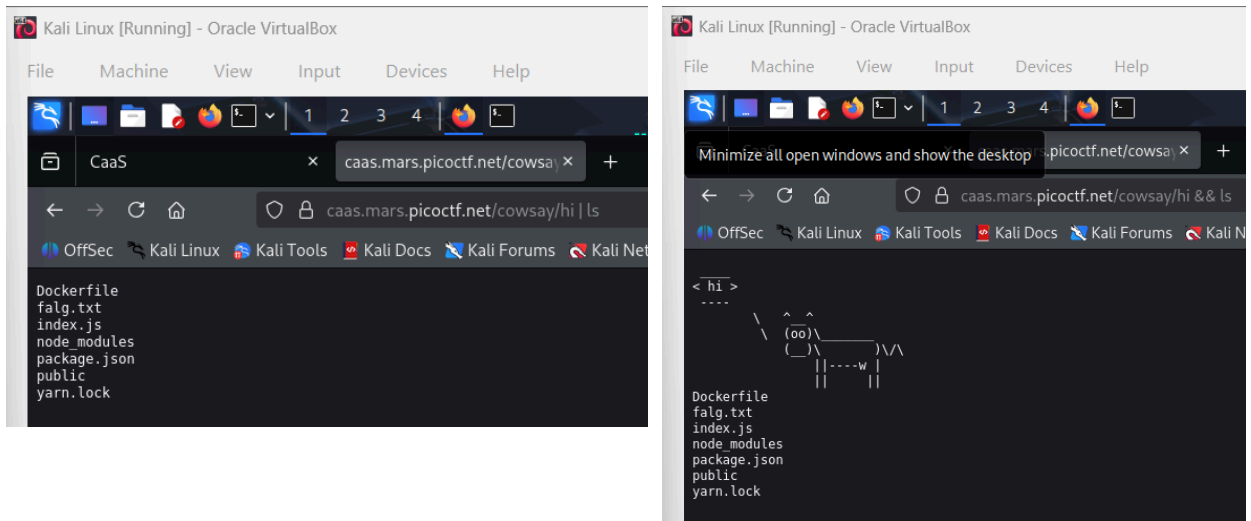
Next, the page source was examined. However, the HTML code did not reveal any useful information about the flag or hidden functionality. Since the challenge involved user input being processed by the server, the possibility of command injection was considered.



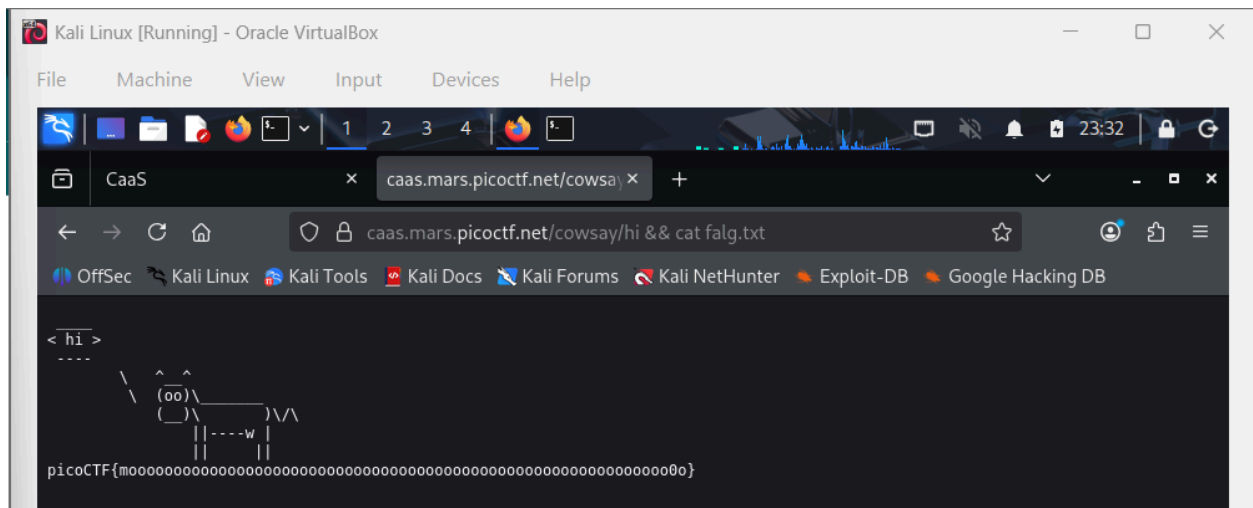
To test this idea, the message parameter in the URL was modified by adding a **semicolon (;)** followed by the command **ls**. The semicolon in command-line syntax allows multiple commands to be executed sequentially, meaning the server might run the original command and then execute the additional command that was injected. After submitting the modified URL, the output displayed a list of files in the server's current directory, confirming that the command injection attempt was successful.



After confirming the vulnerability, additional command operators were tested, such as the **pipe operator (|)** and the **logical AND operator (&&)**, which also executed successfully. Using these operators made it possible to run various commands directly on the server.



With the ability to execute commands, the next step was to locate the flag file. Running the **ls** command revealed several files in the directory, including a file named **flag.txt**. To retrieve its contents, the **cat** command was injected into the URL to display the contents of the file. Once the command was executed, the flag appeared in the output.



caas

Medium

Web Exploitation

picoMini by redpwn

AUTHOR: BROWNIEINMOTION

Description

Now presenting cowsay as a service

Hints ?
(None)

CHALLENGE ENDPOINTS

Download index.js

index.js

22,045 users solved

97% Liked

picoCTF{moooooooooooooooooooooooooooooooooooooo}

Submit Flag

Hurray! You solved this challenge.

Reflection:

This challenge demonstrated how improper handling of user input can lead to command injection vulnerabilities. By injecting command-line operators into the URL parameter, it was possible to execute system commands and access sensitive files such as the flag. The activity emphasized the importance of validating and sanitizing user input to prevent attackers from running unauthorized commands on a server.